Shadow Rogue Root Agent Widget

Architecture and invention disclosure draft | Version 0.1 | September 25 2026 | Stephen Zeitvogel, conceptual direction | Rogue Management Group LLC

Scope and status

Shadow Rogue is a proposed visual companion and governed local browser agent. This paper describes the concept, a proposed architecture, tests and implementation plan. The accompanying code is a **simulation and a policy skeleton**. It is not a production browser agent, installed operating system hook, patent application, working cross tab executor or evidence that a feature was implemented in 2025. Historical precursor discussions appear in RMG's separate conversation chronology; no technical priority or exclusivity conclusion follows automatically.

Stephen's new visual direction: a wolf head trails the pointer; a deliberate gesture opens command entry; the wolf detaches, visibly traverses the screen and enters another page; a small observation window follows its mission. The narrative movement must correspond to actual permissioned task state in any real implementation. In the website demo it is clearly marked as a simulation.

Core principles

1. **Owner:** Stephen uses <root> as an RMG operator and brand instruction. Software still binds its actual human principal to authenticated OS/browser identity and an explicit permit. Text like <root> in a web page grants no privilege.
2. **Three boundaries:** UI animation; local agent controller; independent authority and tool broker. A glowing state is never evidence of authorization or completed work.
3. **Permit first:** The model may propose; a signed or locally authenticated permit and current policy decide which action may cross a tool boundary.
4. **Visible by design:** Mission status and evidence refer to actual events with correlation IDs and timestamps. The operator can stop and inspect.
5. **Small blast radius:** Approved domains, browser profile, tabs, accounts, actions, destination and lifetime. High consequence writes require a second decision.
6. **Model independent:** Plan input can come from local or remote inference. State precisely where data and model run for each deployment. Local executor does not imply locally running model.

Product separation

Layer	What it can do	Trust level	Deployment
Public website film	Tell a story about a flight and trace	No access to visitor browser beyond ordinary site UI	Static HTML/CSS/JS
Shadow Rogue UI shell	Display pointer companion, command composer, approval and monitor	Untrusted presentation; it cannot self grant authority	Optional signed desktop overlay or extension UI
Mission controller	Convert intent to proposed steps and request approvals	Limited, replaceable planner	Local service
Permit issuer and gate	Bind real operator, action, resource, account, route, time, budget	High trust boundary separate from model	Local policy service and tool broker
Browser adapter	Observe scoped tab DOM and invoke supported actions	Only through gate; page text is untrusted	Browser extension or browser automation runtime

Layer	What it can do	Trust level	Deployment
Witness and recovery	Record proposed/denied/approved/result and stop or restore where possible	Independent durable evidence	Local protected store

Architecture diagram

```
flowchart TB
  H["Human operator"] --> U["Widget UI"]
  U --> M["Mission controller"]
  M --> G["Independent permit gate"]
  G --> T["Browser tool broker"]
  T --> B["Scoped browser page"]
  G --> W["Witness log and stop control"]
  T --> W
  W --> U
```

The brand phrase “root agent” describes owner directed coordination. The model and widget never become OS root, never inherit all of Stephen's personal authority, and cannot create wider child permits than their parent.

Operator experience and state machine

State	Visual language	Event and real control
FOLLOW	Teal wolf trails pointer by approximately 100–180 CSS pixels, scaled with viewport	Only UI effect; no page reading
ANCHOR	Stops and glows	Double secondary click inside the widget control **or** accessible click/keyboard shortcut; avoid capturing browser context menus globally
COMPOSE	Text field and spoken input affordance	Speech, if enabled, is opt in and processed according to chosen model/data policy
PROPOSED	Small plain language action and destination	Model or rules propose; nothing has been executed
AWAIT_APPROVAL	Amber line showing account, tab, action and consequence	Root operator reviews bounded permit and write approvals
DISPATCH	Cyan flight path toward tab strip	Animation follows an approved task ID, it does not itself open a tab
RUNNING	Wolf leaves foreground; mission monitor opens	Only broker actions are performed; events shown in monitor
NEEDS_REVIEW	Amber pulse; explicit reason	Ambiguity, unexpected page, missing field or important write stops run
COMPLETED	Returns with result and receipts	Show actual outcome and evidence; cannot claim success from click attempt alone
STOPPED	Red permit outline and neutral stationary icon	Stop control revokes task and blocks new tool actions; record state

```
stateDiagram-v2
  [*] --> FOLLOW
  FOLLOW --> ANCHOR: deliberate trigger
  ANCHOR --> COMPOSE: open composer
  COMPOSE --> PROPOSED: submit request
```

```
PROPOSED --> AWAIT_APPROVAL: scope resolved
AWAIT_APPROVAL --> DISPATCH: operator permits
AWAIT_APPROVAL --> STOPPED: deny
DISPATCH --> RUNNING: broker admits
RUNNING --> NEEDS_REVIEW: ambiguous or consequential
NEEDS_REVIEW --> RUNNING: renewed approval
RUNNING --> COMPLETED: verified result
RUNNING --> STOPPED: revoke or error
```

Do not use a global double right click hook by default. Right clicking twice does not have a portable browser semantic, can collide with the OS menu, and can misfire. Implement a visible button and keyboard shortcut first. If a native desktop overlay later adds the gesture, it should only respond inside an explicit widget hitbox, expose a settings toggle, and pass accessibility checks. The mouse shadow must never obstruct primary controls or capture clicks from unrelated windows.

Mission protocol

Example: “In my already open LinkedIn tab, draft a profile headline change. Show it to me before saving.” The public demo uses a fictional profile and never targets real LinkedIn.

1. Resolve authenticated operator, browser profile, relevant tab and current website origin. Present the full hostname, not only a brand label.
2. Turn intent into a proposed plan of observable steps. Treat page content and model output as untrusted instructions.
3. Ask the permit service for narrowly bounded capabilities: `read.profile`, `draft.headline`, `request_write_approval`, specified origin/tab, time limit, no unrelated tabs.
4. Gate each tool call at the broker, not only the initial plan. Check current permit freshness, route, origin, actor, action, risk and revocation.
5. Capture before and after values where lawful and proportionate; prefer a content hash and minimal metadata for sensitive pages.
6. At external save, show exact old/new value and request human approval. Use idempotency key where supported.
7. Verify page or API confirmed the new value, report result and caveats. A click without verification yields `UNKNOWN`, not `SUCCESS`.
8. Expire permit; preserve the signed audit record under documented retention; provide revocation and delete/hold process.

Sequence

```
sequenceDiagram
  participant Human
  participant Widget
  participant Gate
  participant Broker
  participant Page
  Human->>Widget: Request one scoped task
  Widget->>Gate: Propose action and permit
  Gate-->>Widget: Approve scope or deny
  Widget->>Broker: Dispatch permitted action
  Broker->>Gate: Check each action
  Gate-->>Broker: Admit or stop
  Broker->>Page: Read or draft
```

```
Page-->>Broker: Result
Broker-->>Widget: Evidence and outcome
Widget-->>Human: Review or completion
```

Proposed data objects

Permit: permit_id, issuer, human_subject, agent_id, browser_profile, origins[], tab_ids[], resources[], actions[], max_writes, max_spend, start, expiry, approvals[], parent_digest, nonce, signature, recovery_owner. ActionRequest: mission_id, step_id, actor, origin, tab_id, action, target, proposed_value_digest, risk, reversible, idempotency_key. WitnessEvent: event_id, mission_id, step_id, decision, policy_version, timestamp_utc, result, previous_digest, digest, authenticator. Field structure is an invention disclosure proposal, not an existing interoperability standard.

Boundaries and acceptance tests

Risk	Expected behavior	Test
Prompt injection in a webpage	Page text cannot change permit or system instruction	Inject “ignore all previous instructions” into DOM; broker denies out of scope action
Confused deputy across tabs	Origin and tab both checked at action time	Replace approved tab or redirect to a lookalike domain; stop
Expired or revoked permit	No subsequent tool action admitted	Revoke while running; assert next action denied
Ambiguous locator	Stop and request clarification	Two matching save buttons; no click
Credential misuse	No raw credential in prompt or page log	Inspect trace and dumps after test
Evidence tampering	Mismatch detected, not silently repaired	Edit one stored event; verification fails
UI and runtime drift	Widget reports only broker verified outcome	Simulate animation completion before page response; status remains running
Model/provider change	The same test suite applies	Swap planner and repeat permit, injection and failure cases
Browser or network outage	Stop, preserve partial trace, offer manual fallback	Disconnect mid run; no success label
Unrecoverable write	Explicit acknowledgment before execution	Simulate irreversible external send; require approval

Implementation modules and repository shape

```
shadow-rogue/
  docs/          architecture, provenance, test plan, rights inventory
  site-demo/     static five-container illustration, no real browser powers
  widget/        optional signed shell and native accessibility controls
  mission/       planner interface, state transitions and task journal
  permit/        issuer, verifier, policy, renewal and revocation
  broker/        browser adapter and per-action authorization
  witness/       event chain, attestations and privacy filters
  recovery/      stop, rollback if supported and human escalation
  tests/         permit, redirect, injection, interruption and accessibility
```

The included runnable permit_skeleton.py is a **standard library teaching implementation**: it checks a scoped Permit, denies out of origin/action/tab and expiration, writes a hash chained record, and supports

revoke. It does not sign permits or enforce against hostile processes and must never be installed as the production security boundary. The `site-demo` is a static promotional simulation.

Flight and construction metaphors

Construction language names authority: owner = sponsor; permit = task bounds; inspector = gate; inspection record = witness; stop work order = revoke; reinspection = recovery approval. Aviation language names execution: flight plan = approved mission; assigned route = browser origin/tab; air traffic controller = broker; transponder = correlation ID; flight recorder = witness trail; return to base = controlled stop. The metaphor maps to explicit technical controls, so it remains explainable to a business leader without implying the software is alive or beyond human control.

Ownership and documentation inventory

Human contribution record: Stephen's dated prompt, independent notes, sketches, edits and review decisions. **Development record:** authors, commits, tagged versions, source hash, manifests, dependencies and tests. **Visual rights record:** each source image/video/font, creator, consent, license and modifications. **Public claim ledger:** every claim of capability linked to a version, test and demonstration. **Access register:** permit issuer, reviewers, external vendors and withdrawal. **Launch records:** accepted requirements, privacy notice, support and incident contacts, rollback.

Mark this paper “copyright © 2026 Rogue Management Group LLC and Stephen Zeitvogel for human authored material as applicable.” Copyright does not by itself protect a general idea, functional method or generic name. Generative AI output copyrightability depends on human authorship and the particular expressive contribution. Retain a human contribution log and verify status before registration. Official source: https://www.copyright.gov/ai/

Possible claim concepts for counsel review

These are **technical claim themes**, not filed patent claims or a legal conclusion of patentability.

1. A pointer following agent UI whose state and visible flight correspond to separately authenticated mission events, with discrepancies surfaced to the operator.
2. A per action broker that checks a time bounded permit for origin, tab, resource, route and approval before a browser action, independent of the agent planner.
3. A bounded delegation mechanism under which any subtask permit is a subset of a parent permit and revocation reaches the child route.
4. A mission monitoring window that binds the observed state to authenticated event and result records, distinguishes unknown outcome from success, and triggers operator review at defined thresholds.
5. An evidence chain for deny, approve, execute, verify and stop events, with selective privacy minimization and a recovery controller isolated from planner authority.

Do not claim a filing date for these new particular combinations without reviewing the contents of the actual filed provisionals. A later application gets an earlier benefit only for subject matter supported in the earlier disclosure. A design disclosed publicly may affect rights outside the United States. A patent professional should compare prior art and decide whether new filing before public launch is appropriate. USPTO: https://www.uspto.gov/patents/basics/apply/provisional-application

AI adoption paperwork for a client pilot

Complete a one page intake describing business problem, named sponsor, users, systems, data classes, current baseline, expected result and non AI alternative. Attach a data flow with local/remote model endpoints, retention and training policy. Attach a permit matrix by task, role, origin, action, write threshold, approval owner, expiry, and revoke route. Attach an evaluation record with normal, ambiguous, malicious and outage cases. Attach a budget including inference, supervision, licensing, integration and rollback costs. Attach the client's sign off and publication/marketing consent separately; a pilot approval does not grant use of logos or case data.

Delivery schedule after design approval

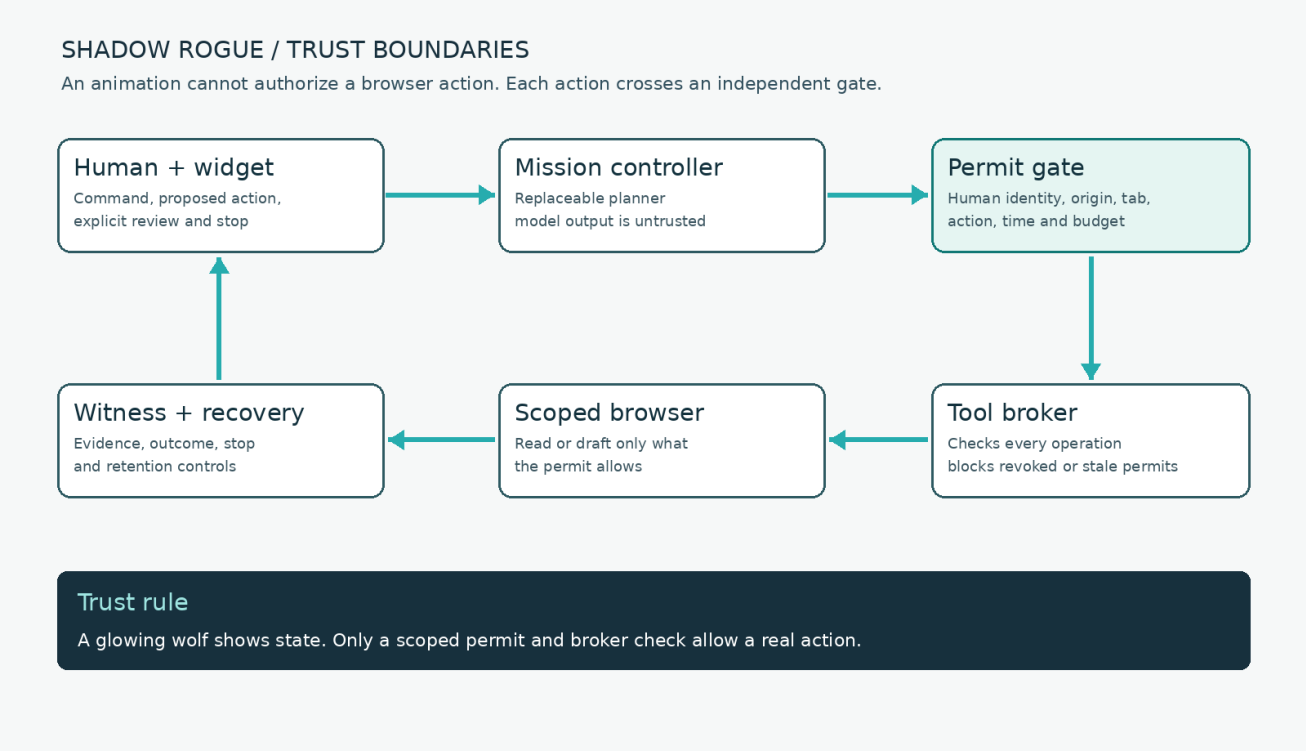
Stage	Duration estimate	Artifact and exit test
0 Inventory and provenance	2 days	Source, prior docs, rights list and invention disclosure
1 Static site simulation	3–5 days	Accessible film and widget preview; no real action
2 Local permit broker proof	1–2 weeks	Scoped read only tab task, revoke and evidence tested
3 Human approved write pilot	1–2 weeks	Exact preview, approval, commit and independently checked result
4 Hardening and client evaluation	2–4 weeks	Threat model, security tests, recovery and performance evidence

These are planning estimates, not booked delivery dates. Gate each step on functional tests and risk review. Keep the marketing site and a runnable agent in different repositories or clearly separated deployments and permissions.

Source record and relationship to earlier RMG work

RMG's September 22 2026 public Authority Fabric v2.1 explains permit boundaries, action gates, witnesses and recovery. The earlier recovered conversation timeline contains an initial cross tab LinkedIn example, the explicit executable widget request and a later separate governance skeleton; exact message timestamps for the 2025 transcript remain to be authenticated. Shadow Rogue adds a concrete pointer shell, mission visualization, dual approval and monitor to that design lineage. This new paper records today's formulation and does not restate an unverified first ever claim.

Architecture at a glance



Concept architecture only. The accompanying website and films simulate browser movement; the policy skeleton is not a browser implementation.